

EV-AUT-03 — Portafolio de Evidencias Individuales

Autoevaluación sumativa final del periodo

Iván Andrés Villamarín Cuenca

Ingeniería de Software, Universidad Técnica Estatal de Quevedo (UTEQ)

Aplicaciones Distribuidas (ISR-701), Equipo PFC: ForáCode

iavillamarin98-pred

Resumen—Este portafolio documenta tres evidencias de aprendizaje individual y mi contribución verificable al Proyecto Fin de Curso (PFC) del sistema SCLI (Sistema de Control de Laboratorios e Informática), desarrollado por el equipo ForáCode. Presento código propio de autenticación JWT, un análisis técnico de tolerancia a fallos sobre un clúster CockroachDB interpretado con el teorema CAP y el algoritmo de consenso Raft, y una reflexión crítica sobre un error de redes en Docker Compose, fundamentada en literatura académica independiente. Todo lo declarado es rastreable hasta commits específicos en los dos repositorios del proyecto.

Index Terms—autoevaluación, portafolio de evidencias, autenticación JWT, tolerancia a fallos, CAP, Raft, Docker, microservicios.

I. INTRODUCCIÓN

Este documento cierra mi evaluación del periodo mediante un portafolio personal de evidencias verificables, conforme al instrumento EV-AUT-03. Todo lo que declaro aquí es rastreable hasta commits específicos en los dos repositorios del proyecto: el repositorio de la Entrega 3 (Proyecto_Fin_de_Curso_scli) y el repositorio de la Entrega 4, actualmente en desarrollo (Entrega-final-del-PFC).

II. PARTE (A): EVIDENCIAS DE APRENDIZAJE

A. Evidencia 1 — Código propio: autenticación JWT stateless

Unidad del curso: Aplicaciones Distribuidas — Autenticación y seguridad en arquitecturas de microservicios (Entrega 3 del PFC).

Artefacto: `auth-service/src/main/java/.../SecurityConfig.java`, `AuthController.java`, `JwtService.java`. Commit `2da67b2` (repositorio Proyecto_Fin_de_Curso_scli, rama villamarin, 14 de julio de 2026).

Explicación del aprendizaje:

Al construir el `auth-service` consideré el manejo de sesiones mediante Spring Security como alternativa, pero opté por JWT porque es una solución más adecuada para una

Repositorio del portafolio: github.com/ivillamarinc/EV-SUM-04-portafolio-villamarin. Repositorio del PFC (E3): github.com/iavillamarin98-pred/Proyecto_Fin_de_Curso_scli. Repositorio del PFC (E4): github.com/ffarinangog2/Entrega-final-del-PFC. Periodo 2026–2027. Manuscrito recibido el 21 de agosto de 2026.

arquitectura de microservicios. En este tipo de sistemas distribuidos, mantener sesiones en memoria en un único servidor introduce un punto de acoplamiento y un posible cuello de botella, además de complicar la escalabilidad horizontal. Elegí JWT porque permite un enfoque *stateless*, donde cada token contiene de forma firmada la información mínima necesaria para autenticar y autorizar al usuario, lo que elimina la dependencia de un almacenamiento central de sesiones.

A diferencia de las sesiones tradicionales, que requieren consultas constantes a un servidor o a un almacén compartido como Redis, con JWT logro que cada solicitud sea autónoma, ya que el token puede validarse localmente mediante la firma criptográfica. Esto mejora el rendimiento y reduce la latencia en sistemas distribuidos, además de facilitar la interoperabilidad entre microservicios, ya que cualquiera de ellos puede verificar la autenticidad del token sin necesidad de comunicarse con el `auth-service`.

Fundamenté el uso de JWT en el estándar abierto RFC 7519, lo que me garantiza compatibilidad y buenas prácticas en su implementación. Su estructura basada en *header*, *payload* y *signature* me permitió incluir *claims* como roles o identificadores de usuario, facilitando la implementación de autorización basada en roles dentro del ecosistema de microservicios. Por estas razones técnicas y arquitectónicas, implementé JWT como el mecanismo principal de autenticación del sistema.

B. Evidencia 2 — Análisis técnico: tolerancia a fallos y consenso distribuido

Unidad del curso: Aplicaciones Distribuidas — Bases de datos distribuidas, consenso y tolerancia a fallos (Entrega 3 del PFC).

Artefacto: `docs/evidencias/tolerancia_fallos.md`, commit `4430e86` (28 de julio de 2026). Fundamento teórico: commit `a79167f` (`docs/main.tex`, 29 de julio de 2026). Ambos en el repositorio Proyecto_Fin_de_Curso_scli, rama `feature/entrega-3-villamarin`.

El artefacto:

Explicación del aprendizaje:

Diseñé y ejecuté una prueba de tolerancia a fallos sobre el clúster CockroachDB de tres nodos que desplegamos para el microservicio de reservas y solicitudes. La prueba consistió en cinco pasos: medir la latencia de un `SELECT COUNT (*)`

Cuadro I: Latencia de `SELECT COUNT (*)` por escenario de disponibilidad del clúster CockroachDB (n=1 por escenario).

Escenario	Resultado	Latencia
Clúster operativo (3 nodos)	Exitosa	2604.54 ms
Un nodo fuera de servicio	Exitosa	2681.40 ms
Clúster recuperado (3 nodos)	Exitosa	2906.34 ms
Dos nodos fuera de servicio	Sin disponibilidad	No aplica

sobre 100 000 filas con el clúster completo funcionando; apagar abruptamente un nodo con `docker stop` y repetir la consulta; volver a encender el nodo y medir de nuevo una vez que se recuperó; y por último repetir todo apagando dos de los tres nodos. Cada latencia que reporto en la Tabla I es de una sola medición por escenario (n=1), no un promedio de varias repeticiones — lo dejo claro porque el protocolo completo del curso pide 10 repeticiones con intervalo de confianza al 95%, y esa parte todavía no la hicimos; presentar una sola medición como si fuera una serie completa de datos habría sido deshonesto.

Interpreto estos resultados con el teorema CAP y su versión más completa, PACELC. CockroachDB prioriza consistencia sobre disponibilidad, tanto cuando hay una falla de red como en operación normal. Esto se explica por el algoritmo¹ de consenso que usa, llamado Raft: cada porción de datos² del clúster forma su propio grupo de réplicas, y ese grupo³ solo puede confirmar escrituras (o, en este caso, responder⁴ lecturas de forma confiable) si la mayoría de sus réplicas están⁵ disponibles. Con tres réplicas en total, la mayoría necesaria es de 2: por eso el clúster aguantó la caída de un nodo sin perder disponibilidad (2 de 3 réplicas seguían siendo mayoría), pero⁶ sí perdió disponibilidad al caer dos nodos, porque ya ninguna⁷ combinación de réplicas alcanzaba esa mayoría.⁸

Lo que más me hizo pensar de estos datos fue algo que⁹ no esperaba: la latencia **después** de recuperar el clúster¹⁰ (2906.34 ms) fue mayor que la latencia con un nodo caído¹¹ (2681.40 ms), en vez de volver al valor original (2604.54 ms).¹² Mi primera intuición fue pensar que era ruido de mi propio¹³ equipo. Pero al pensarlo con el marco de Raft, tiene una ex¹⁴ plicación consistente: cuando el nodo detenido se reincorpora¹⁵ no vuelve a estar disponible de inmediato — el líder del¹⁶ grupo Raft primero tiene que ponerlo al día con todos los cambios que se hicieron mientras estuvo caído, antes de que pueda volver a responder consultas de forma confiable. Ese proceso de ponerse al día añade una carga extra justo en el momento en que hice la medición. No puedo afirmar con una sola observación que esto se deba completamente a Raft y no a variabilidad de mi propio equipo — para eso necesitaría las repeticiones que el protocolo completo exige. Esta cautela no es solo mía: Kleppmann [1] muestra que el teorema CAP en su forma clásica es más ambiguo de lo que parece, y que pensar en términos binarios de “consistente” o “disponible” oculta matices reales de latencia que una sola medición no puede capturar. Que la latencia suba justo después de recuperar el

nodo, y no antes, coincide con lo que la teoría de Raft predice, y es la limitación más honesta que puedo señalar sobre mi propio dato.

Una precisión de autoría que quiero dejar explícita: los datos de la tabla y el fundamento teórico de CAP/Raft son de mi autoría (commits 4430e86 y a79167f). La redacción final de la Sección 5.7 del informe grupal de Entrega 3, que integra estos datos en la narrativa consolidada, fue escrita por mi compañero Isaías Urbina (commit 3ad41b2) a partir de mi evidencia. Lo declaro así porque prefiero un aporte modesto correctamente atribuido a uno inflado.

C. Evidencia 3 — Reflexión documentada: healthchecks y redes en Docker Compose

Unidad del curso: Aplicaciones Distribuidas — Contenerización, orquestación y despliegue de microservicios (Entrega 4 del PFC).

Artefacto: `docker-compose.yml`, `api-gateway/.../GatewayRoutes.java` y `application.yml`. Commit 4f95c43 (repositorio Entrega-final-del-PFC, rama feature/entrega-4, 19 de agosto de 2026).

El artefacto (fragmento representativo):

Listing 1: Cambio en las rutas del API Gateway (commit 4f95c43).

```
# Antes
- id: auth-service
  uri: http://localhost:8081

# Despues
- id: auth-service
  uri: ${AUTH_SERVICE_URL:http://auth-service:8081}
```

Listing 2: Condicion de dependencia y healthcheck agregado.

```
# Antes
depends_on:
  auth-service:
    condition: service_started

# Despues
depends_on:
  auth-service:
    condition: service_healthy
healthcheck:
  test: ["CMD-SHELL", "wget -q -O /dev/null http://localhost:8081/actuator/health || exit 1"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 60s
```

Reflexión:

Cuando armé el `api-gateway` para que hablara con los demás microservicios, escribí las URLs de conexión como `http://localhost:8081`, etc. Fuera de Docker eso funcionaba perfecto en mi máquina, así que asumí sin cuestionarlo que iba a funcionar igual dentro de los contenedores. Estaba equivocado: dentro de la red de Docker Compose, `localhost` desde el contenedor del gateway apunta al propio contenedor del gateway, no al contenedor de `auth-service`. El gateway nunca podía comunicarse con los otros servicios cuando todo corría dockerizado, y al principio no entendía por qué, porque en mi computadora “sí

funcionaba” — solo que estaba probando cada servicio suelto, no dentro del compose completo.

El segundo supuesto que tenía mal fue asumir que si un contenedor ya estaba “levantado” (`service_started`), el servicio de adentro ya podía recibir peticiones. En la práctica, un contenedor puede arrancar en segundos mientras que la aplicación Spring Boot de adentro tarda bastante más en terminar de inicializar. El gateway a veces arrancaba antes de que `auth-service` estuviera realmente listo, y fallaban las primeras conexiones de forma intermitente.

Corregí ambas cosas: cambié las URLs fijas por variables de entorno que usan el nombre del servicio de Docker Compose como dirección, porque Docker resuelve esos nombres automáticamente dentro de su propia red. Y le agregué un healthcheck a cada microservicio, para que Docker supiera distinguir entre “el contenedor existe” y “el servicio de adentro ya puede responder”.

Lo que me queda de esto es que dentro de un sistema de contenedores el estado de un proceso no es algo que se pueda dar por hecho desde afuera — hay que definirlo explícitamente con un healthcheck, porque si no, el orquestador solo sabe si el contenedor existe, no si el servicio de adentro realmente funciona. No fue solo un error mío de principiante: Barletta et al. [2] documentan que las fallas de conectividad entre servicios representan una fracción medible y recurrente de los fallos reales en sistemas orquestados con Kubernetes, lo que sugiere que el supuesto que yo tenía — que un contenedor “arriba” ya puede recibir tráfico — es un error común, no una particularidad de mi implementación. Esto también me hace pensar de forma más crítica sobre mi propia solución: agregar `start_period: 60s` y `retries: 5` resuelve el síntoma que yo observé con mi volumen de datos y mi máquina, pero es un valor fijo que asumí, no uno que medí sistemáticamente — Chen et al. [3] muestran que el comportamiento de resiliencia ante fallos de red varía bastante según el entorno de despliegue, así que no puedo asumir que ese mismo valor funcionaría igual de bien en otra infraestructura sin volver a probarlo.

Es una lección que no es específica de Docker Compose: se aplicaría igual si en algún momento usamos Kubernetes u otro orquestador, porque el mismo problema — contenedor “arriba” pero servicio no listo todavía — existe en cualquier sistema que orqueste servicios interdependientes. Si volviera a armar esto desde cero, definiría los healthchecks desde el principio del proyecto, no como una corrección que llegó casi al final.

III. PARTE (B): CONTRIBUCIÓN INDIVIDUAL AL PFC

Mis 25 commits verificados en el PFC se distribuyen entre el 11 de julio y el 19 de agosto de 2026, cubriendo 8 fechas distintas de trabajo real, no concentradas en las horas previas a una entrega. De estos, 12 son de código e implementación (`auth-service`, `api-gateway`, `frontend`, `pipeline Spark`), 9 de documentación técnica (LaTeX del informe, README, referencias) y 4 de infraestructura (Docker, configuración). Esta mezcla y distribución muestran un aporte sostenido a lo largo del periodo, no una carga masiva de último momento.

A. Evidencia del historial (capturas)

B. Tabla de commits

Nota de coautoría: el commit 3ad41b2 (redacción final de la Sección 5.7 del informe E3, que integra los datos de mi Evidencia 2) fue escrito por mi compañero Isaías Urbina, no por mí. Lo declaro explícitamente para distinguir mi aporte real del de mi equipo.

IV. DECLARACIÓN DE USO DE IA GENERATIVA

Para la elaboración del presente informe se utilizó una herramienta de inteligencia artificial como apoyo en la organización de ideas, la corrección de ambigüedades, la revisión gramatical conforme a la rúbrica EV-AUT-03, la explicación de conceptos sobre las herramientas del PFC y la orientación en la comparación de alternativas. Asimismo, se empleó en la verificación técnica de los commits de los repositorios Git (Proyecto_Fin_de_Curso_scli y Entrega-final-del-PFC). La selección de las fuentes bibliográficas, la revisión de su pertinencia y la verificación de referencias en repositorios académicos (ResearchGate, Google Scholar y otros) fueron realizadas manualmente por el autor. La selección de evidencias, las decisiones de diseño, el análisis, la interpretación de los resultados y la postura crítica presentados son de autoría individual del estudiante, quien revisó y editó el contenido en su totalidad antes de la entrega.

REFERENCIAS

- [1] M. Kleppmann, *A Critique of the CAP Theorem*, sep. de 2015. DOI: [10.48550/arXiv.1509.05393](https://doi.org/10.48550/arXiv.1509.05393) arXiv: [1509.05393](https://arxiv.org/abs/1509.05393) [cs.DC]. dirección: <https://arxiv.org/abs/1509.05393>
- [2] M. Barletta, M. Cinque, C. D. Martino, Z. T. Kalbarczyk y R. K. Iyer, “Mutiny! How Does Kubernetes Fail, and What Can We Do about It?” En *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, IEEE, 2024, págs. 1-14. DOI: [10.1109/DSN58291.2024.00016](https://doi.org/10.1109/DSN58291.2024.00016) arXiv: [2404.11169](https://arxiv.org/abs/2404.11169) [cs.DC]. dirección: <https://arxiv.org/abs/2404.11169>
- [3] Z. Chen, M. Goudarzi y A. N. Toosi, *Resilience Evaluation of Kubernetes in Cloud-Edge Environments via Failure Injection*, jul. de 2025. DOI: [10.48550/arXiv.2507.16109](https://doi.org/10.48550/arXiv.2507.16109) arXiv: [2507.16109](https://arxiv.org/abs/2507.16109) [cs.DC]. dirección: <https://arxiv.org/abs/2507.16109>

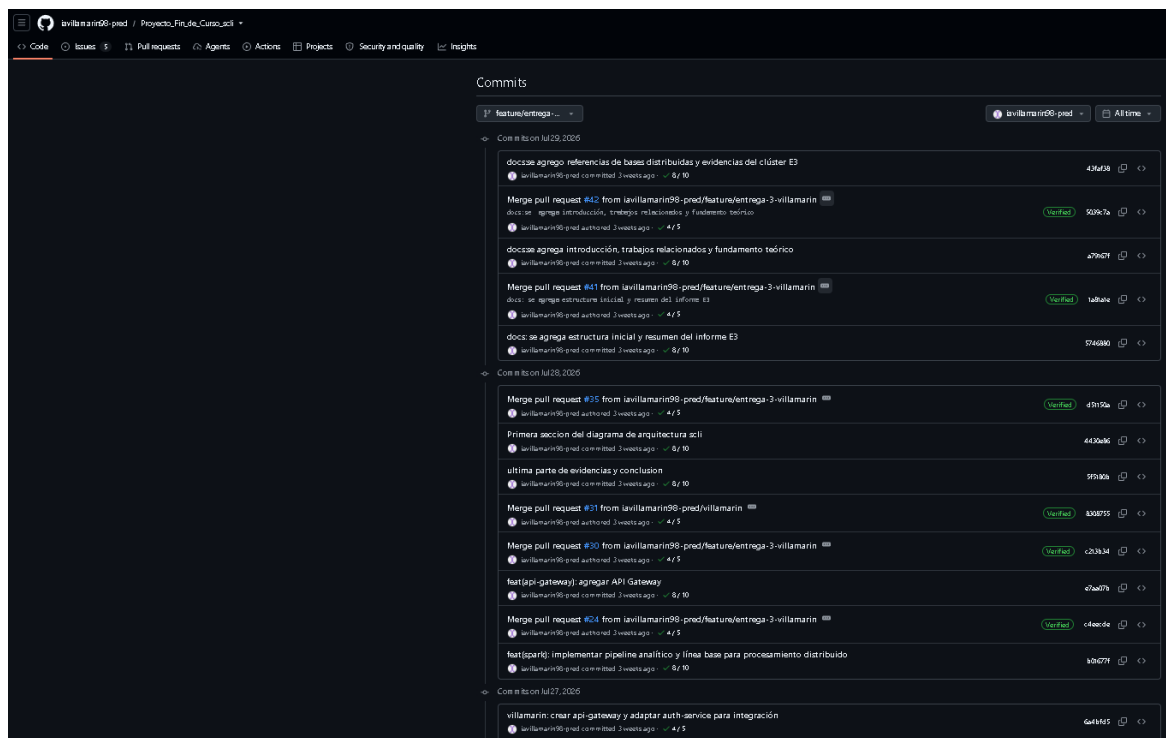


Figura 1: Historial de commits filtrado por autoría en el repositorio Proyecto_Fin_de_Curso_scli, rama feature/entrega-3-villamarin.

URL reproducible:

https://github.com/iavillamarin98-pred/Proyecto_Fin_de_Curso_scli/commits/feature/entrega-3-villamarin?author=iavillamarin98-pred

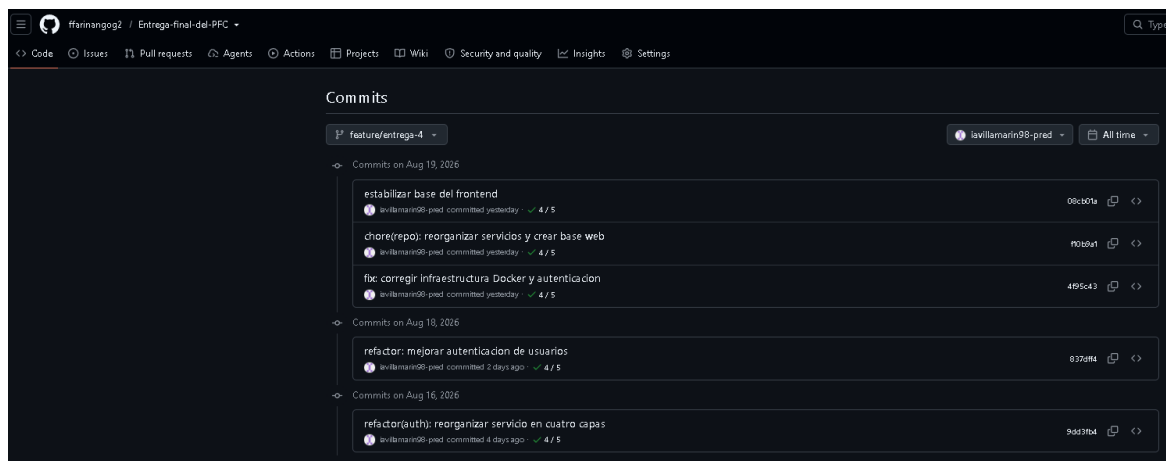


Figura 2: Historial de commits filtrado por autoría en el repositorio Entrega-final-del-PFC, rama feature/entrega-4.

URL reproducible: <https://github.com/ffarinangog2/Entrega-final-del-PFC/commits/feature/entrega-4?author=iavillamarin98-pred>

Cuadro II: Los 25 commits verificados de mi autoría en ambos repositorios del PFC.

Hash	Fecha	Tipo	Mensaje	Archivos principales
<i>Repositorio: Proyecto_Fin_de_Curso_scli (rama feature/entrega-3-villamarin)</i>				
98db210	11 jul	Infra	chore: initialize SCLI microservices architecture	docker-compose.yml
e810092	11 jul	Docs	Create README.md	usuarios-service/README.md
d5936e4	12 jul	Docs	Create README.md	auth-service/README.md
58539fa	12 jul	Código	Avances del auth-service	Entidades JPA, SecurityConfig, Flyway V1-V3
6c82617	12 jul	Docs	Create README.md	reservas-solicitudes-service/README.md
d024158	12 jul	Docs	Create README.md	academico-laboratorios-service/README.md
2da67b2	14 jul	Código	feat(auth): implementa JWT y endpoints	SecurityConfig.java, AuthController.java — Ev. 1
8a6720a	14 jul	Código	feat(frontend): agrega scli-front	Frontend Vue completo
43f7fc4	20 jul	Código	integra autenticación y agrega dashboard	DashboardView.vue, auth.ts
263db06	20 jul	Infra	fix(usuarios): usa JRE Alpine	usuarios-service/Dockerfile
dbc46fe	20 jul	Docs	elimina README obsoleto de la raíz	README.md
833cff9	27 jul	Código	integrar auth-service con usuarios-service	AuthService.java, migraciones SQL
6a4bfd5	27 jul	Código	crear api-gateway y adaptar auth-service	api-gateway/ completo
b01677f	28 jul	Código	feat(spark): pipeline analítico y línea base	spark/pipeline.py, spark/baseline.py
e7aa07b	28 jul	Docs	feat(api-gateway): agregar API Gateway	api-gateway/HELP.md
5f5180b	28 jul	Datos	última parte de evidencias y conclusión	tolerancia_fallos.md
4430e86	28 jul	Datos	Primera sección del diagrama de arquitectura	arquitecturaScli.drawio, tolerancia_fallos.md — Ev. 2
5746880	29 jul	Docs	estructura inicial y resumen del informe E3	docs/main.tex
a79167f	29 jul	Docs	introducción, trabajos relacionados y fundamento teórico	docs/main.tex — Ev. 2
43faf38	29 jul	Docs	referencias de bases distribuidas y evidencias	Referencias.bib, imágenes
<i>Repositorio: Entrega-final-del-PFC (rama feature/entrega-4)</i>				
9dd3fb4	16 ago	Código	refactor(auth): reorganizar en cuatro capas	Capas presentation/application/domain/infra
837dff4	18 ago	Código	refactor: mejorar autenticación de usuarios	Auth service
4f95c43	19 ago	Infra	fix: corregir infraestructura Docker y autenticación	docker-compose.yml, GatewayRoutes.java — Ev. 3
f10b9a1	19 ago	Infra	chore(repo): reorganizar servicios y crear base web	Reestructuración de carpetas
08cb01a	19 ago	Código	estabilizar base del frontend	Frontend web